

Penetration Testing Report

Kuber Kapuriya
Computer Security CS 419

1 Executive Summary

My testing covered five categories: authentication, authorization, input validation, session security, and configuration. I used both automated test coverage (147 python test scripts) and manual testing with curl, PowerShell, and my browser.

21/21 checklist items are completed with no current issues. The only item with some confusion is password reset security because I didn't implement self service password reset/forgot password and instead have authenticated change password when a user is already logged in. The project description didn't mention specifics for this so I went with the more secure ability to change password while already logged in instead of creating larger attack surface by letting any when use a "forgot password" button in login screen. Three findings were identified during testing: Finding 1 - an information disclosure in the Werkzeug generated **Server** header (cant fix at the application layer). Finding 2 - a low severity gap where the `/register` endpoint previously didn't use rate limiting. Finding 3 - low severity limitation where the per IP rate limiter does not defend against distributed attacks from multiple source IPs. I accepted this risk because its not a publicly deployed app, if it was publicly deployed severity would be medium or high. Overall, no critical, high, or medium severity issues were discovered. The application's core security controls like authentication, role-based authorization, encryption at rest, and transport security are performed as designed in every test.

2 Testing Methodology

In my testing I combined three approaches: automated testing with the project's existing pytest scripts (147 tests across six files), targeted manual testing against a running instance of the application on localhost, and direct inspection of HTTP requests and responses using browser developer tools and curl. Places where an automated test is used to prove a checklist item, I cite the specific test class and name as evidence rather than describing all the test logic.

For each checklist item, I attempted to exercise the control as an attacker would. For example, brute forcing credentials, crafting malformed filenames, attempting to view another user's documents by ID manipulation, and submitting input designed to bypass validation. I compared the expected result from the Security Design Document against the observed behavior and any mismatches were recorded here as findings.

3 Test Checklist

A. Authentication Testing

- ✓ Brute force protection : covered by `test_auth.py::TestLogin::test_login_rate_limit`, `test_account_lockout_after_five_failures`, `test_failed_attempts_incremented`, `test_locked_until_is_float_timestamp`, and `test_locked_message_contains_minutes`. Rate limiting returns 429 after 10 attempts per IP per 60 seconds on both `/login` and `/register` (registration limiting was added later). Account lockout starts after 5 failed logins for 15 minutes.

- ✓ Password complexity enforcement : covered by `test_input_validation.py::TestValidatePassword` (8 tests): minimum 12 characters, required uppercase, lowercase, digit, and special character. Boundary cases at exactly 12 characters checked. Registration flow covered by `test_auth.py::TestRegistration::test_weak_password_rejected`.
- ✓ Session management : covered by `test_session.py::TestSessionManagerUnit` (9 tests) and `TestSessionIntegration` (8 tests): token creation, validation, destruction, expiration, server side revocation, invalid token rejection, and cookie flag verification against the live Set-Cookie header (HttpOnly and SameSite=Strict checked).
- ✓ Logout functionality : covered by `test_auth.py::TestLogout::test_logout_clears_session`, `test_logout_flash_message`, and `test_session.py::TestSessionIntegration::test_logout_removes_server_session`. Logout deletes the server side session record, not just the cookie.
- ✓ Password reset security : Doesn't apply: As previously mentioned, unauthenticated self service reset is not implemented. Authenticated password change is available through the account page and reuses the existing `@require_auth` guard, `validate_password` complexity rules, and bcrypt hashing.

B. Authorization Testing

- ✓ Horizontal privilege escalation : covered by `test_access_control.py::TestDocumentOwnership::test_non_owner_cannot_view`, `test_non_owner_cannot_delete`, `TestSharing::test_download_unshared_document`, and `test_audit.py::TestDocumentAuditRoute::test_non_owner_non_admin_denied`. Nonowners receive 403 on view, delete, download, and audit routes.
- ✓ Vertical privilege escalation : covered by `test_access_control.py::TestAdminAccess::test_regular_user_cannot_access_admin`, `TestGuestPermissions` (6 tests: upload, version, delete, admin access, and the editor to viewer share downgrade), and `test_audit.py::TestAdminAuditFilter::test_non_admin_cannot_access`. Role enforcement is decorator based and server side only.
- ✓ Direct object reference (IDOR) : covered by `test_access_control.py::TestDocumentOwnership::test_non_owner_cannot_view`, `TestSharing::test_download_unshared_document`, and `test_missing_doc_returns_404`. Requests for documents owned by another user return 403. nonexistent IDs return 404.
- ✓ Forced browsing : I verified manually using chrome developer tools and curl. A logged in guest session was used to issue direct requests to 11 sensitive routes like `/admin`, `/admin/audit`, `/upload` (GET and POST), and `/document/nonexistent/{audit,delete,version}`. The cookie was confirmed valid by a 200 response on `/dashboard`, then every privileged route returned either 403 or a 404 for nonexistent resources. No 200 on any route a guest should not reach, and no 500 showing an exception leaking past the access control check. GET requests on nonexistent document IDs returned 404, while POST requests returned 403. Both behaviors are consistent with the `owns_document` and `@deny_guest` guards I described in my Security Design Document. Logout returned 302. `test_access_control.py::TestUnauthenticatedAccess` gives more coverage for the unauthenticated case.

C. Input Validation Testing

- ✓ XSS attempts : covered by `test_input_validation.py::TestSanitizeInput` (4 tests), `TestFormInputs::test_xss_in_username_not_reflected_raw`, `test_oversized_username_rejected`, and `TestFileUploadIntegration::test_xss_in_filename`. Payloads in usernames and filenames are escaped and not reflected as raw HTML.
- ✓ Path traversal : covered by `test_input_validation.py::TestSafeFilename::test_path_traversal_stripped`, `TestSafeFilePath::test_traversal_raises`, `TestFileUploadIntegration::test_path_traversal_in_filename`, and `test_path_traversal_in_download`. Traversal payloads are stripped or rejected at both the filename and path resolution layers.
- ✓ Command injection : verified manually. Used case insensitive Select-String powershell command on `app.py`, `security.py`, and `config.py` for `os.system`, `subprocess`, `eval()`, `exec()`, and `render_template_string` returned zero matches. This confirmed the codebase contains no shell execution or template string with user input. Payload testing confirmed the expected behavior: a registration attempt with a semicolon and whoami sequence in the username field returned 200, and a login attempt with a command substitution payload(whoami) in the username field returned 200. Neither response contained command output.
- ✓ File upload vulnerabilities : covered by `test_input_validation.py::TestFileUploadIntegration::test_file_extension_whitelist`, `test_oversized_file`, `test_xss_in_filename`, `test_path_traversal_in_filename`, and `test_encryption.py::TestUploadIntegration::test_uploaded_file_stored_encrypted`. Extension whitelist and 16MB cap enforced. Files are stored as Fernet ciphertext with no plaintext leakage.

D. Session Security Testing

- ✓ Session fixation : verified manually with 3 step attack simulation. (Step 1) A login request was made while presenting an attacker chosen cookie value (`session_token=attacker_known_token`). The server's Set-Cookie response returned a completely different server generated token (exact token was `_fat_YsviEteocYJqc_1LKfMsZVrfZ-WDWlrPmiEb1w`), confirming that already existing client tokens are ignored at login. (Step 2) The returned cookie carried the expected flags: `Secure`, `HttpOnly`, `SameSite=Strict`, and `Max-Age=1800`. (Step 3) A follow up request to `/dashboard` using the attacker's original token returned 302 (redirect to login), confirming that planted token never became a valid session. Combined with auto test script `test_session.py::TestSessionIntegration::test_invalid_token_rejected`, this establishes that session tokens are issued server side on every login and cannot be preset by an attacker.
- ✓ Session hijacking : covered by `test_session.py::TestSessionIntegration::test_server_side_revocation_denies_cookie`, `test_expired_session_denied`, and `test_invalid_token_rejected`. Clearing the server side session store invalidates any cookie immediately. Expired sessions are denied on the next request.
- ✓ Concurrent session handling : covered by `test_session.py::TestSessionIntegration::test_concurrent_sessions`. Two simultaneous logins for the same user produces two independent session tokens. Destroying one does not effect the other.
- ✓ Session timeout : covered by `test_session.py::TestSessionManagerUnit::test_expired_session_returns_none`, `test_validate_updates_last_activity`, `test_cleanup_removes`

expired, and `TestSessionIntegration::test_expired_session_denied`. Idle timeout is 30 minutes and absolute lifetime is 8 hours enforced by `validate_session` on every request.

E. Configuration Testing

- ✓ Security headers : verified manually with `curl.exe-k-Ihttps://localhost:5000/login`. I didn't use `securityheaders.com` because it seems it need a public site. All expected headers are present: `Strict-Transport-Security` (`max-age=31536000andincludeSubDomains`), `Content-Security-Policy` (with `default-src'self'` and `frame-ancestors'none'`), `X-Frame-Options:DENY`, `X-Content-Type-Options:nosniff`, `Referrer-Policy:strict-origin-when-cross-origin`, and `Permissions-Policy` denies geolocation, microphone, and camera. The known CSP relaxation (`'unsafe-inline'` in `style-src`) is present. There is one finding for security headers that I report in findings.
- ✓ TLS/SSL configuration : verified manually across three checks. (Check 1) HTTP requests on port 5000 are rejected at the transport layer (`curl:(56)Recvfailure:Connectionwasreset`). Flask only listens for TLS, so plaintext HTTP cannot be served accidentally. (Check 2) Certificate inspection (`opensslx509`) confirms `sha256WithRSAEncryption` signature, RSA 2048-bit public key, self-signed with `Subject==Issuer==CN=localhost`, and a 365 day validity window (Apr 2 2026 to Apr 2 2027). The self signed cert is a limitation I mentioned in the Security Design Document and is acceptable for only localhost use. (Check 3) TLS handshake with `--tlsv1.2` succeeds and returns `HTTP/1.1200OK`. TLS 1.1 is rejected at the client by Windows Schannel (`SEC_E_UNSUPPORTED_FUNCTION`), so legacy TLS versions are not in use on either side of the connection.
- ✓ Error handling : verified manually across 403, 404, 429, and 500 handlers. `Select-String` command of `app.py` confirms registered `@app.errorhandler` decorators for all four codes. A request to a nonexistent route returned the 404 page with only a generic title, user message, and navigation link. No stack trace, Python paths, framework version, exception type, or config data in the response body. Unauthenticated access to `/admin` was redirected to `/login` rather than returning 403. The 403 handler fires only for authenticated but unauthorized access, which is verified by `test_access_control.py::TestAdminAccess::test_regular_user_cannot_access_admin`. The 429 rate limit handler was confirmed by making 12 rapid login POSTs from the same IP: the first 10 returned 200, the last 2 returned 429. I didn't try to trigger a deliberate 500 since that would happen when an unhandled Python exception escapes a route handler.
- ✓ Debug mode disabled : verified manually. The `app.run()` call in `app.py` passes `debug=False`. A case insensitive `Select-String` for debug across `app.py` and `config.py` returns only that single line. No `FLASK_DEBUG` or `FLASK_ENV` environment variables are set when running the app, so the default `ENV='production'` in `config.py` applies. So debugger and stack trace display are disabled.

4 Findings

4.1 F1: Server banner leaks framework and runtime versions

Severity: Very Low

Category E: Configuration Testing (Security headers).

The app responds to every request with a Server header containing the exact versions of Werkzeug and Python in use: `Server:Werkzeug/3.1.3Python/3.13.3`. After eesearching, I discovered this

header is set by Werkzeug's development server at the transport layer and cannot be overridden from the Flask application layer.

Impact: Information disclosure. An attacker who can reach the application can read the Server header without authentication and then look up known CVEs affecting those exact Werkzeug and Python versions to prioritize targeted exploits. Minimal risk because it depends on those versions of Werkzeug and Python to have known vulnerabilities that can be exploited.

Steps to reproduce:

1. Start the application with `python app.py`
2. From a second shell, request any route with a HEAD request: `curl.exe-k-Ihttps://localhost:5000/login`.
3. Look at the **Server:** line in the response headers.

Proof of concept:

Server: Werkzeug/3.1.3 Python/3.13.3

Remediation: Override the Server header from the Flask `after_request` hook. I tried this but it did not work. The correct fix requires deploying the application behind a production WSGI server with a reverse proxy configured to rewrite or hide the Server header. Not needed since project not public.

4.2 F2: Registration endpoint lacked rate limiting

Severity: Medium

Category A: Authentication Testing (Brute force protection, registration endpoint)

In an earlier version of the application, it applied the per IP rate limiter only to the `/login` endpoint. The `/register` endpoint was unprotected. This meant an attacker could script an unlimited number of registration attempts from a single IP.

Impact: There are two ways to exploit this. First, mass account creation bloats the encrypted `users.json` store: because `load_users()` is called on every authenticated request (to check `is_active`), a bloated user file slows read performance across the entire app, making a denial of service condition affecting all users rather than just the registration endpoint. Second, username enumeration where repeated registration attempts with candidate usernames return differentiated error messages like `username taken` vs. `validation failures`, which an unauthenticated attacker could use to identify valid account names. The Severity medium because the could be easily vulnerable to DOS with lack of registration rate limiting.

Steps to reproduce (before I fixed it)

1. Script N sequential `POST/register` requests from a single IP.
2. See that all N requests return 200 regardless of N, with no 429 response ever issued.

Proof of concept (before fix) A PowerShell loop issuing 20 rapid `POST/register` requests received 200 on every attempt, with no limiting.

Remediation (applied) The existing `RateLimiter` class in `security.py` was extended to cover `/register` in addition to `/login`, using the same per IP sliding window. Exceeding the limit now

returns HTTP 429, same as with the login endpoint's behavior. The fix was verified manually by using curl for repeated rapid registration attempts from the same IP.

4.3 F3: Rate limiter is per IP only and does not defend against distributed attacks

Severity: Very Low (Medium for public deployment)

Category A: Authentication Testing (Brute force protection)

The RateLimiter class tracks requests in an in memory sliding window keyed by client IP. An attacker controlling multiple source IPs like a small botnet can distribute requests across those IPs so that no single IP ever exceeds the 10 per 60 seconds threshold. Per account lockout (5 failed logins, 15 minute lockout) still protects any single target account from credential brute force, but endpoint wide abuse like mass registration, username enumeration across many candidate names, or credential stuffing against many usernames at once is not meaningfully slowed.

Impact: In the localhost class project deployment there is no real exposure since the app is not reachable from any other IP, and a single tester cant meaningfully distribute requests. In a public deployment the impact would be rated Medium: an attacker with a many IPs at disposal could do registration abuse, username enumeration, or credential stuffing indefinitely.

Steps to reproduce: No reproduction is possible against a localhost only app.

Remediation. A production deployment could address this by placing the application behind a third party service that handles rate limiting before traffic reaches the app. Something like Cloudflare can apply network edge rate limits across all source IPs, and a CAPTCHA on /login and /register to stop automated request volume regardless of source distribution. No need to implement remediation for local project.

5 Conclusion

Penetration testing confirmed that SecureWebApp's main security controls function as designed, with all 147 automated tests passing and manual verification checks for session fixation, command injection, security headers, and TLS configuration. 3 findings were identified: F1 (Very Low), a Server header leak from the Werkzeug layer that cant be patched from the Flask app and is accepted as a risk. F2 (medium), a missing rate limit check on /register that was discovered during pre testing review and fixed by extending the existing RateLimiter to cover registration. F3 (Low) potentially medium in deployment, the inability of the per IP rate limiter to defend against distributed attacks, accepted as a risk with a clear fix with cloudflare/captcha. The previously documented limitations from the Security Design Document (plaintext logs, SameSite=Strict-only CSRF protection, self signed TLS, SSD storage persistence) remain. Overall, the app shows a good security posture for current project localhost deployment.